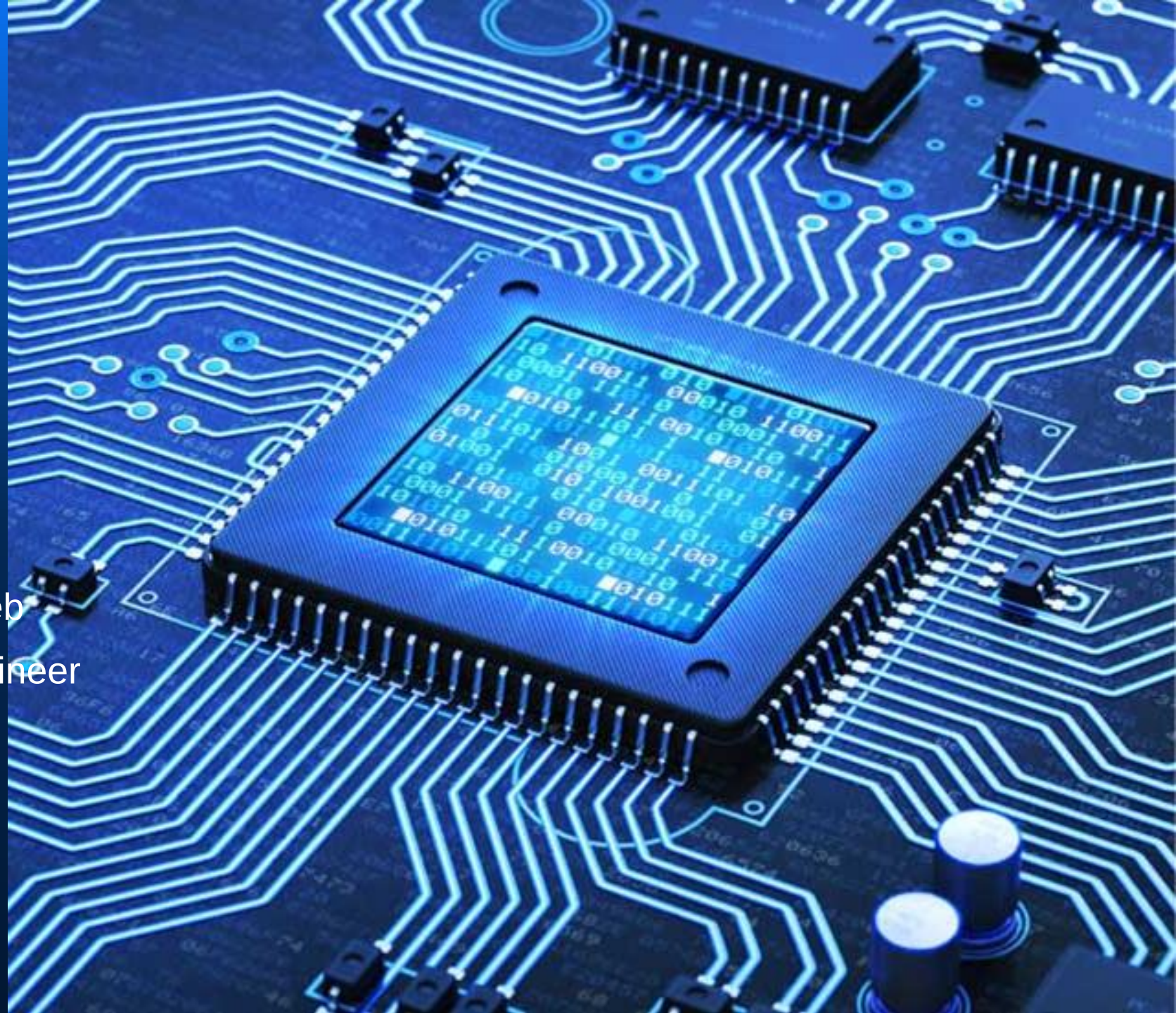
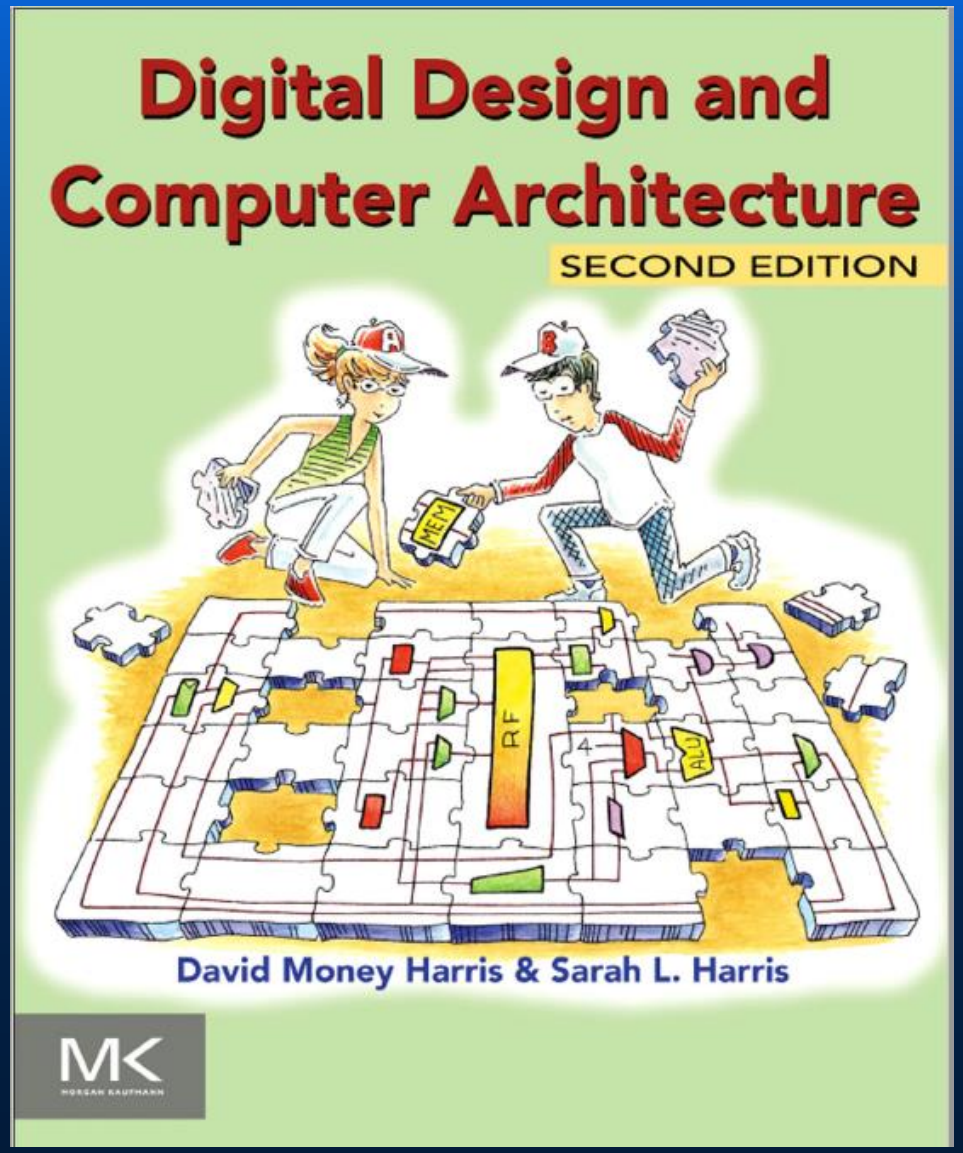
Computer Architecture Session 5

By: Eng. Abdulkareem Abotaleb

Senior HW Digital Design Engineer





This presentation is prepared from:

Digital Design and Computer Architecture, 2nd Edition.
David Harris and Sarah L. Harris.

Agenda

Chapter 7: Microarchitecture

7. Advanced Microarchitecture

7. Advanced Microarchitecture

7. Advanced Microarchitecture

- High-performance microprocessors use a wide variety of **techniques** to run programs faster.
- Every 2 to 3 years, advances in CMOS manufacturing reduce **transistor dimensions** by 30% in each direction, doubling the number of transistors that can fit on a chip.
- Smaller transistors are faster and consume less power. So, the **clock frequency** can increase because all the gates are faster, even if the hardware does not change.
- Smaller transistors enable placing more transistors on a chip. Microarchitects use the additional transistors to build more complicated processors on a chip.
- Unfortunately, power consumption increases with the number of transistors and the operate speed.
- Microprocessor designers should handle the trade-offs among speed, power, and cost for chips with billions of transistors in some of the most complex systems.

7. Advanced Microarchitecture

7.1 Deep Pipelines

7.2 Branch Prediction

7.3 Superscalar Processor

7.4 Out-of-Order Processor

7.5 Register Renaming

7.6 Single Instruction Multiple Data

7.7 Multithreading

7.8 Homogeneous Multiprocessors

7.9 Heterogeneous Multiprocessors

7.1 Deep Pipelines

- The easiest way to speed up the clock is to divide the pipeline into more stages.
- Each stage contains less logic, so it can run faster.
- We considered a classic five-stage pipeline, but 10 to 20 stages are now commonly used.
- The max number of pipeline stages is limited by pipeline hazards, sequencing overhead, and cost.
- Longer pipelines introduce more dependencies. Some of the dependencies can be solved by forwarding, but others require stalls, which increase the CPI.
- The pipeline registers between each stage have sequencing overhead from their setup time and clk-to-Q delay (as well as clock skew).
- Adding more stages increases the cost because of the extra pipeline registers and hardware required to handle hazards.

7. Advanced Microarchitecture

7.1 Deep Pipelines

7.2 Branch Prediction

7.3 Superscalar Processor

7.4 Out-of-Order Processor

7.5 Register Renaming

7.6 Single Instruction Multiple Data

7.7 Multithreading

7.8 Homogeneous Multiprocessors

7.9 Heterogeneous Multiprocessors

7.2 Branch Prediction

- The branch misprediction penalty is a major reason for increased CPI.
- As pipelines get deeper, branches are resolved later in the pipeline.
- Thus, the branch misprediction penalty gets larger, because all the instructions issued after the mispredicted branch must be flushed.
- To address this problem, most pipelined processors use **a branch predictor** to guess whether the branch should be taken.
- Recall that our pipeline simply predicted that branches are never taken.

7.2 Branch Prediction

- The simplest form of branch prediction checks the direction of the branch
- **Backward** branches (for or while loops) tend to be executed many times (usually taken).
- This is called **static branch prediction**, because it does not depend on the history of the program.
- **Forward** branches use the history of program execution to guess whether a branch should be taken (**dynamic branch predictors**).
- Dynamic branch predictors maintain a table of the last several hundred (or thousand) branch instructions that the processor has executed.
- The table, sometimes called **a branch target buffer**, includes the destination of the branch and a history of whether the branch was taken.

7.2 Branch Prediction

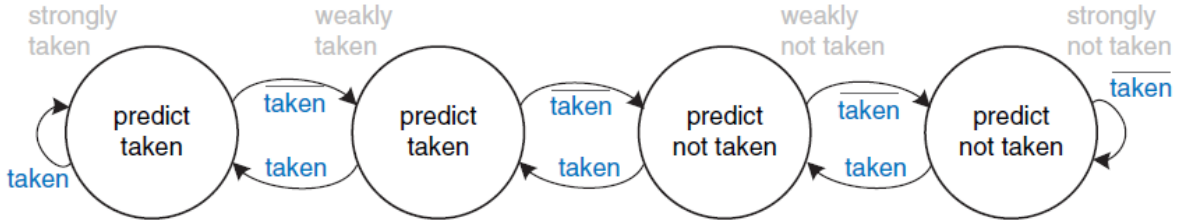
- **1-bit dynamic branch predictor**
  stores whether the branch was taken the last time
  predicts that it will do the same thing the next time.
- During loop, branch is not taken till last time
  and predicts that it should not be taken next time.
- This is a correct prediction until the last branch of the loop, when the branch gets taken.
- If the loop is run again, the branch predictor remembers that the last branch was taken. So, it predicts the branch should be taken when the loop is first run again.

```
add $s1, $0, $0        # sum = 0
addi $s0, $0, 0        # i = 0
addi $t0, $0, 10       # $t0 = 10
for:
beq $s0, $t0, done     # if i== 10, branch to done
add $s1, $s1, $s0      # sum = sum + i
addi $s0, $s0, 1       # increment i
j for
done:
```

- 1-bit branch predictor mispredicts the first and last branches of a loop.

7.2 Branch Prediction

- **2-bit dynamic branch predictor** solves this problem by having four states: strongly taken, weakly taken, weakly not taken, and strongly not taken.
- During loop, it enters the “strongly not taken” state and predicts that the branch should not be taken.
- This is correct until the last branch of the loop, which is taken and moves the predictor to the “weakly not taken” state.



- If the loop is run again, the branch predictor correctly predicts that the branch should not be taken and reenters the “strongly not taken” state.
- 2-bit branch predictor mispredicts only the last branch of a loop.

7.2 Branch Prediction

- Branch predictors may be used to track even more history of the program to increase the accuracy of predictions.
- Good branch predictors achieve better than 90% accuracy on typical programs.
- The branch predictor operates in **the Fetch stage** of the pipeline so that it can determine which instruction to execute on the next cycle.
- When it predicts that the branch should be taken, the processor fetches the next instruction from the branch destination stored in **the branch target buffer**.
- By keeping track of both branch and jump destinations in the branch target buffer, the processor can also avoid flushing the pipeline during jump instructions.

7. Advanced Microarchitecture

7.1 Deep Pipelines

7.2 Branch Prediction

7.3 Superscalar Processor

7.4 Out-of-Order Processor

7.5 Register Renaming

7.6 Single Instruction Multiple Data

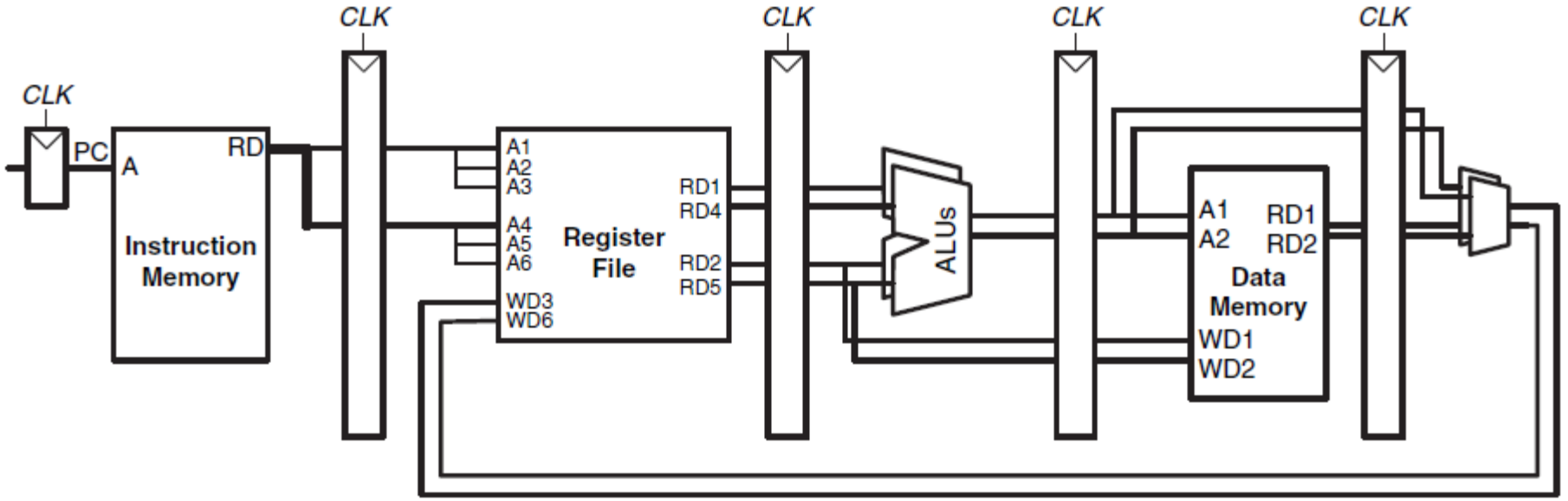
7.7 Multithreading

7.8 Homogeneous Multiprocessors

7.9 Heterogeneous Multiprocessors

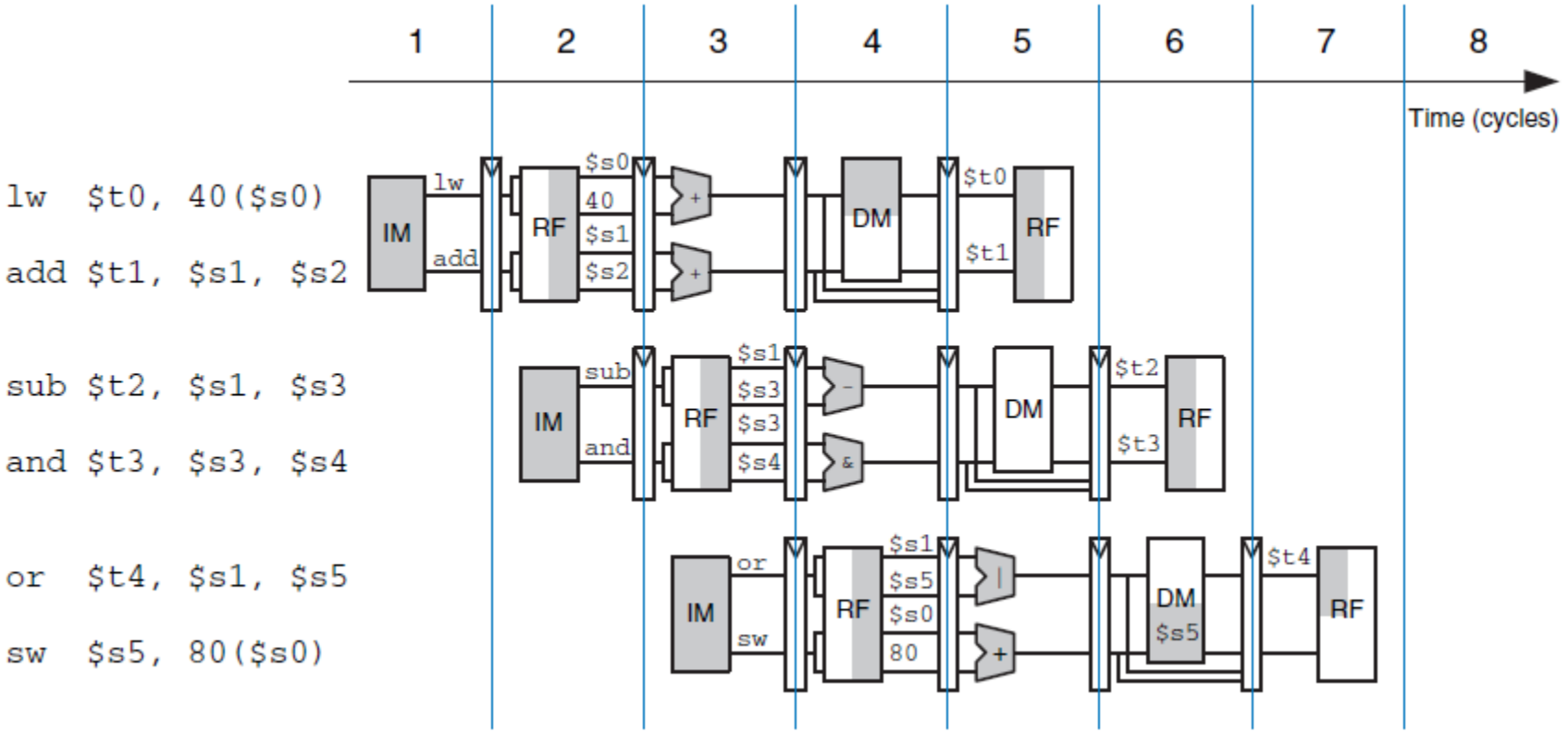
7.3 Superscalar Processor

- A superscalar processor contains multiple copies of the datapath hardware to execute multiple instructions simultaneously.
- The datapath fetches two instructions at a time from the instruction memory.
- It has a six-ported register file to read 4 source operands and write 2 results back in each cycle.
- It also contains 2 ALUs and a 2-ported data memory to execute the 2 instructions at the same time.



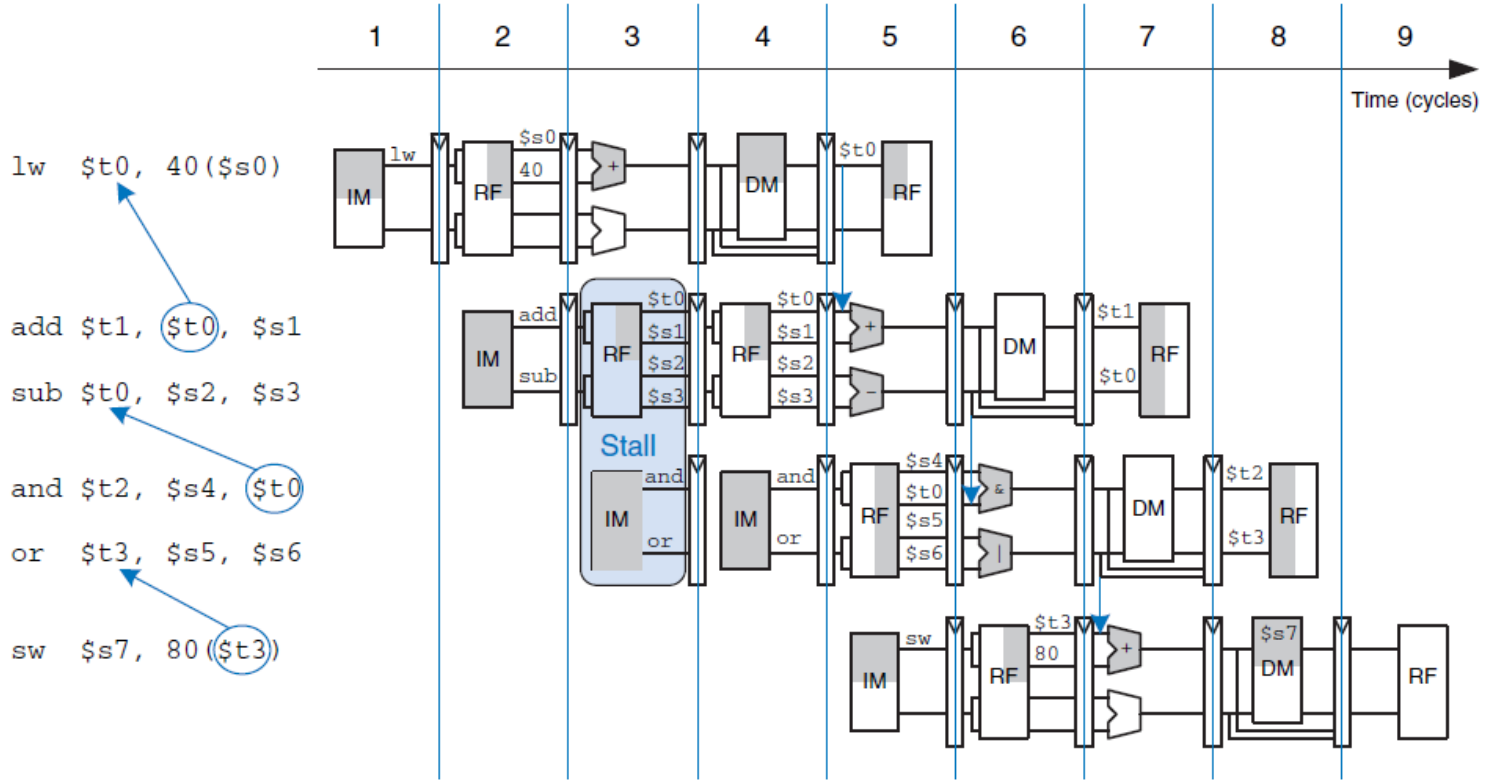
7.3 Superscalar Processor

- The pipeline shows the two-way superscalar processor executing 2 instructions on each cycle.
- For this program, the processor has a CPI of 0.5 (or IPC of 2).



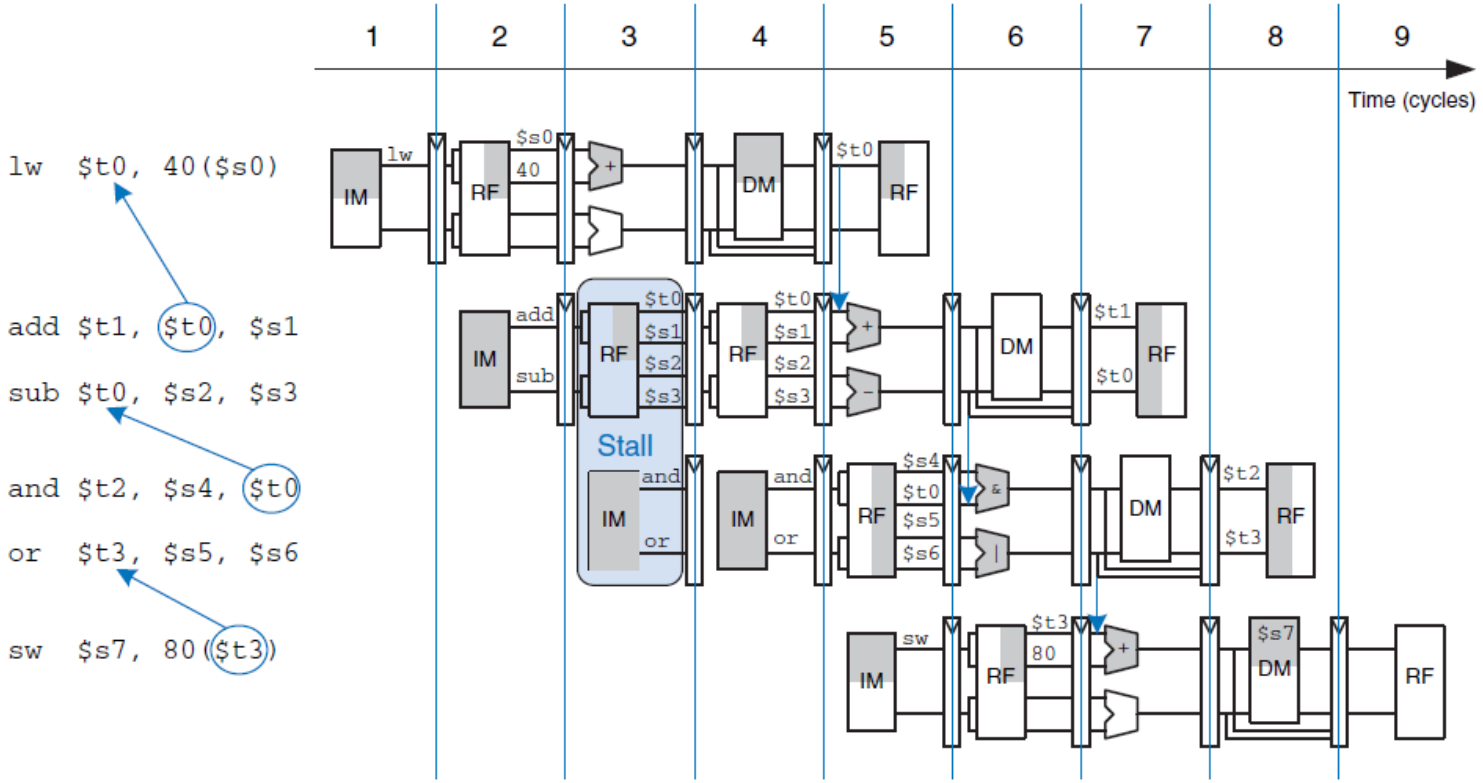
7.3 Superscalar Processor

- Executing many instructions simultaneously is difficult because of dependencies.
- Example: The **add** instruction is dependent on $t0, which is produced by the **lw** instruction, so it cannot be issued at the same time as lw.
- The **add** instruction stalls for yet another cycle so that **lw** can forward $t0 to add in cycle 5.



7.3 Superscalar Processor

- The other dependencies (between **sub** and **and** based on $t0, and between **or** and **sw** based on $t3) are handled by forwarding results produced in one cycle to be consumed in the next.
- This program requires 5 cycles to issue 6 instructions, for an IPC of 1.17.



7.3 Superscalar Processor

- Superscalar processors utilize the temporal parallelism through pipelining and the spatial parallelism through multiple execution units to enhance the performance.
- Commercial processors may be three-, four-, or even six-way superscalar.
- They must handle control hazards such as branches as well as data hazards.
- Superscalar processors rarely fully utilize all execution units, because of the dependencies.
- The large number of execution units and complex forwarding networks consume vast amounts of area and power.

7. Advanced Microarchitecture

7.1 Deep Pipelines

7.2 Branch Prediction

7.3 Superscalar Processor

7.4 Out-of-Order Processor

7.5 Register Renaming

7.6 Single Instruction Multiple Data

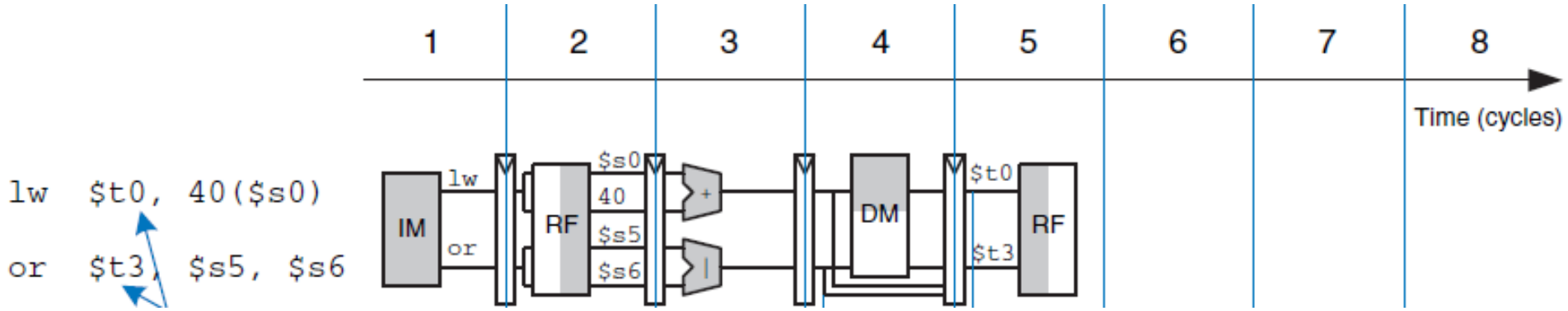
7.7 Multithreading

7.8 Homogeneous Multiprocessors

7.9 Heterogeneous Multiprocessors

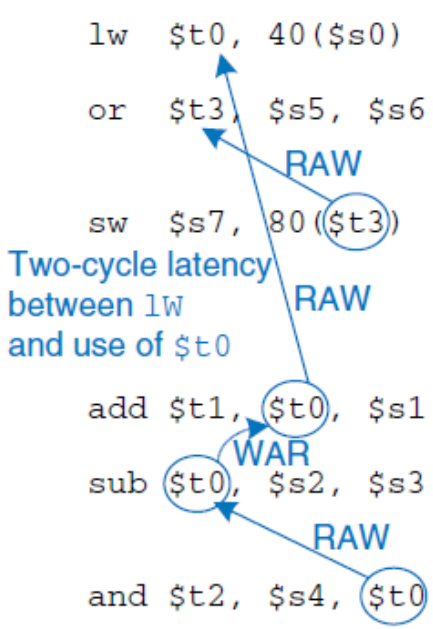
7.4 Out-of-Order Processor

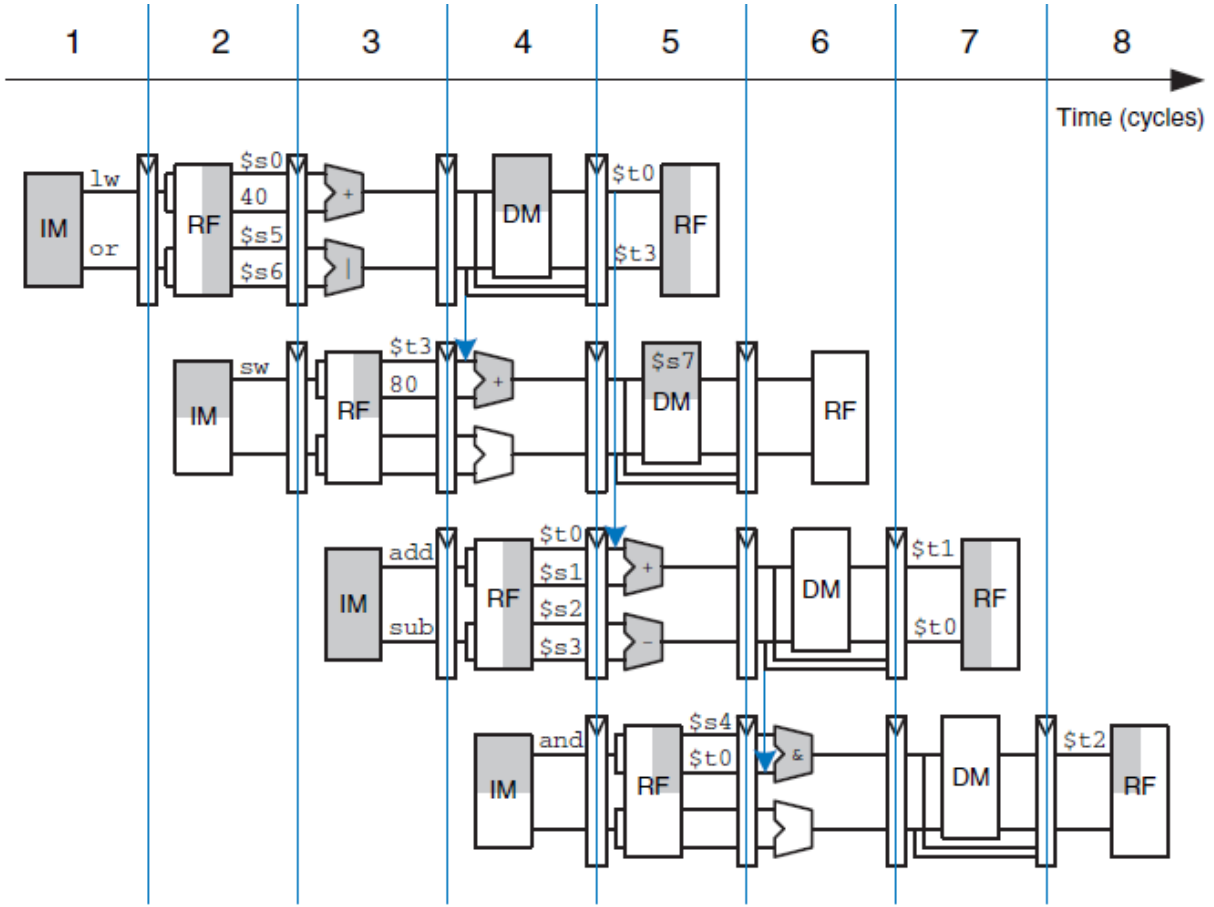
- The out-of-order processor executes independent instructions as rapidly as possible.
- The instructions can be issued in a different order than that written by the programmer, as long as the program produces the intended result.
- Example: Cycle 1
  - The **lw** instruction issues.
  - The **add**, **sub**, and **and** instructions are dependent on **lw** by way of $t0, so they cannot issue yet.
  - The **or** instruction is independent, so it also issues.



7.4 Out-of-Order Processor

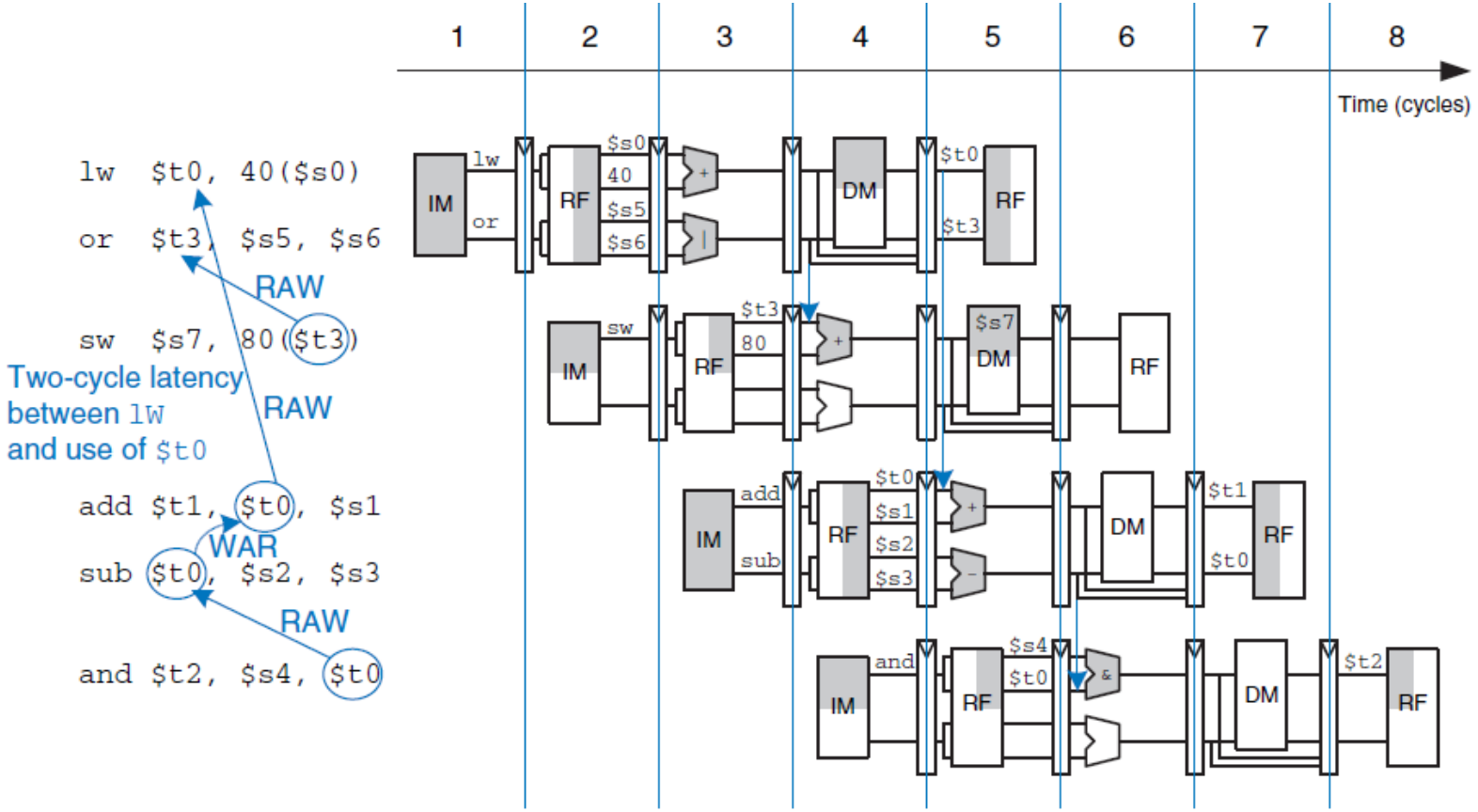
- Cycle 2
  - **add** cannot issue yet because of the $t0 dependence.
  - **sub** writes $t0, so it cannot issue before **add**, otherwise **add** receive the wrong value of $t0.
  - **and** is dependent on **sub**.
  - Only **sw** instruction issues.





7.4 Out-of-Order Processor

- Cycle 3: **add** issues as $t0 is available. **sub** issues also, as it will write $t0 after **add** uses it.
- Cycle 4: **and** instruction issues. $t0 is forwarded from **sub** to **and**.
- The out-of-order processor issues the 6 instructions in 4 cycles, for an IPC of 1.5.



7.4 Out-of-Order Processor

The read after write (RAW) hazard

- It is shown in the **$t0** dependence of **add** on **lw**, the **$t3** dependence of **sw** on **or** and the **$t0** dependence of **and** on **sub.**

The write after read (WAR) hazard

- It is shown in the **$t0** dependence between **sub** and **add**.
  sub must not write **$t0** before **add** reads **$t0**, so that **add** receives the correct value according to the original order of the program.
- WAR hazards could not occur in the simple MIPS pipeline, but they may happen in an out-of-order processor (if **sub** is moved too early).
- A WAR hazard is caused by the programmer's using the same register for two unrelated instructions. If the **sub** instruction had written $t4, **sub** could be issued before **add**.
- The MIPS architecture only has 32 registers, so sometimes the programmer is forced to reuse a register and introduce a hazard.

7.4 Out-of-Order Processor

The write after write (WAW) hazard

- It occurs if an instruction attempts to write a register after a subsequent instruction has already written it, writing the wrong value to the register.
- Example: The **add** and **sub** both write $t0. The final value in $t0 should come from **sub** according to the order of the program. If an out-of-order processor attempted to execute **sub** first, a WAW hazard would occur.

```
add $t0, $s1, $s2
sub $t0, $s3, $s4
```

- WAW hazards are caused by the programmer's using the same register for two unrelated instr.
- If the **sub** instruction were issued first, the program could eliminate the WAW hazard by discarding the result of the **add** instead of writing it to $t0. This is called **squashing** the **add**.

7.4 Out-of-Order Processor

- Out-of-order processors use a table (a scoreboard) to keep track of instructions waiting to issue.
- The table contains information about the dependencies.
- The size of the table determines how many instructions can be considered for issue.
- On each cycle, the processor examines the table and issues as many instructions as it can, limited by the dependencies and by the number of available execution units (e.g., ALUs, memory ports).
- **The instruction level parallelism (ILP)** is the number of instructions that can be executed simultaneously for a particular program and microarchitecture.
- Theoretical studies have shown that the **ILP** can be quite large for out-of-order microarchitectures with perfect branch predictors and huge number of execution units.
- However, practical processors hardly achieve an **ILP** greater than 2 or 3, even with six-way superscalar datapaths with out-of-order execution.

7. Advanced Microarchitecture

7.1 Deep Pipelines

7.2 Branch Prediction

7.3 Superscalar Processor

7.4 Out-of-Order Processor

7.5 Register Renaming

7.6 Single Instruction Multiple Data

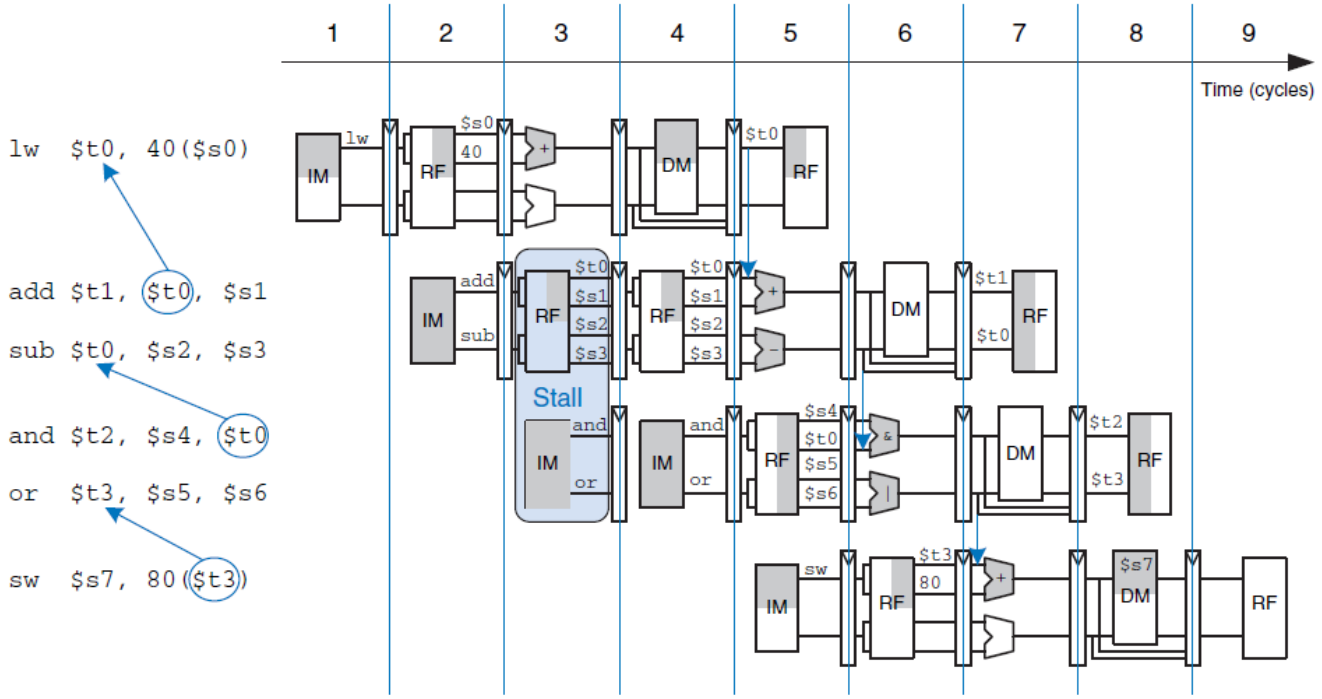
7.7 Multithreading

7.8 Homogeneous Multiprocessors

7.9 Heterogeneous Multiprocessors

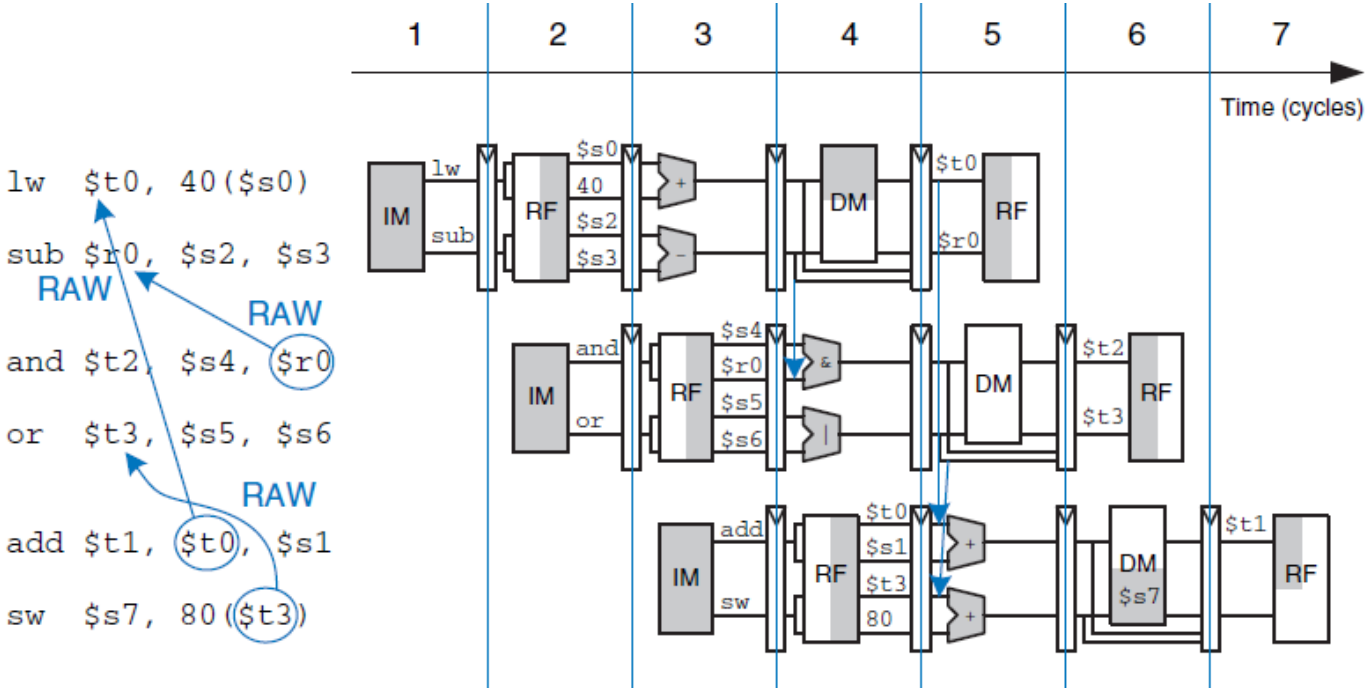
7.5 Register Renaming

- Register renaming is used to eliminate WAR hazards.
- Register renaming adds some nonarchitectural renaming registers to the processor.
- MIPS processor might add 20 renaming registers, called $r0-$r19. The programmer cannot use these registers directly, because they are not part of the architecture.
- Example: a WAR hazard occurred between the **sub** and **add** instructions based on reusing $t0.
- The out-of-order processor could rename $t0 to $r0 for the sub instruction.
- Then **sub** could be executed sooner, because $r0 has no dependency on the **add** instruction.
- $t0 must also be renamed to $r0 in the **and** instruction, as it refers to the result of **sub**.



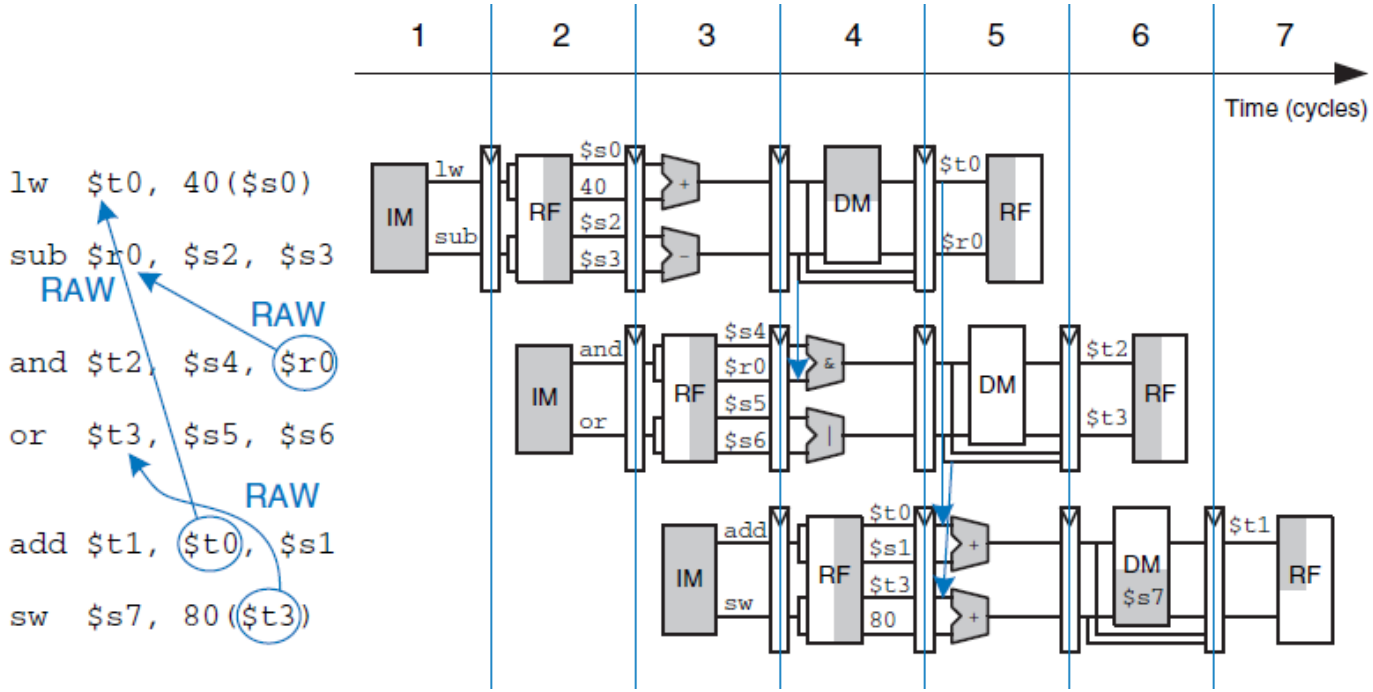
7.5 Register Renaming

- The processor keeps a table of which registers were renamed so that it can consistently rename registers in subsequent dependent instructions.
- $t0 is renamed to $r0 in **sub** and **and** to eliminate the WAR hazard.
- Cycle 1
  - The **lw** instruction issues.
  - The **add** is dependent on **lw**, so it can't issue. However, the **sub** is independent so it also issues.



7.5 Register Renaming

- Cycle 2
  - The **add** cannot issue yet because of the $t0 dependence with **lw** instruction.
  - The **and** instruction is dependent on **sub**, so it can issue. $r0 is forwarded from **sub** to **and**.
  - The **or** instruction is independent, so it also issues.
- Cycle 3
  - The **add** issues as $t0 is available. The **sw** issues as $t3 is also available.
- The out-of-order processor with register renaming issues the 6 instructions in 3 cycles with an IPC of 2.



7. Advanced Microarchitecture

7.1 Deep Pipelines

7.2 Branch Prediction

7.3 Superscalar Processor

7.4 Out-of-Order Processor

7.5 Register Renaming

7.6 Single Instruction Multiple Data

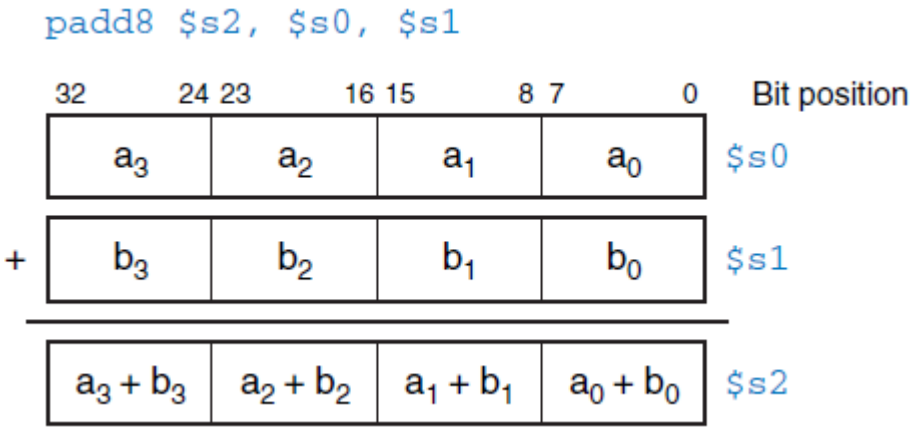
7.7 Multithreading

7.8 Homogeneous Multiprocessors

7.9 Heterogeneous Multiprocessors

7.6 Single Instruction Multiple Data

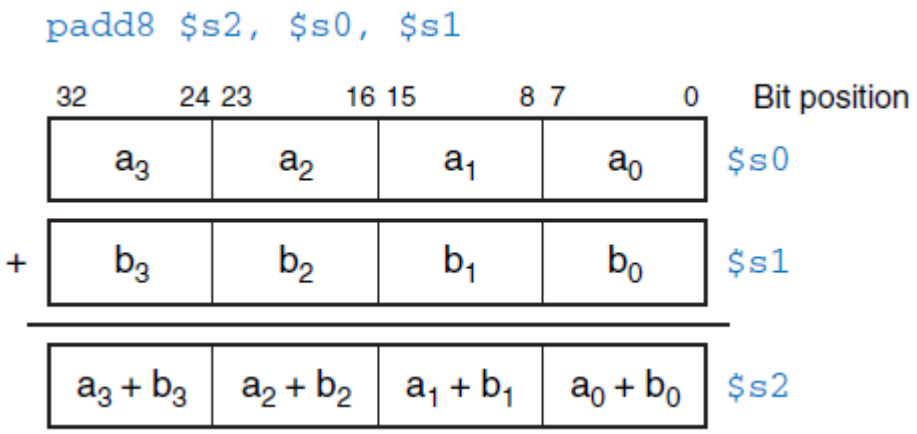
- The SIMD (single instruction multiple data): a single instruction acts on multiple data in parallel.
- A common application of SIMD is to perform many short arithmetic operations at once, especially for graphics processing (packed arithmetic).
- Example: a 32-bit microprocessor might pack four 8-bit data elements into one 32-bit word.
- Packed add and subtract instructions operate on all four data elements within the word in parallel.
- A packed 8-bit addition summing four pairs of 8-bit numbers to produce four results.
- The word could also be divided into two 16-bit elements.



7.6 Single Instruction Multiple Data

Example:

- Compare between expressing the following addition using SIMD padd8 instruction and the traditional instructions:



- Using SIMD padd8 instruction:

```
lui $s0, {a3,a2}
ori $s0, {a1,a0}
lui $s1, {b3,b2}
ori $s1, {b1,b0}
padd8 $s2, $s1, $s0
```

```
addiu $t0, $0, a0
addiu $t1, $0, b0
add $s2, $t0, $t1
;=====
addiu $t0, $0, a1
addiu $t1, $0, b1
add $t2, $t0, $t1
sll $t2, $t2, 8
or $s2, $s2, $t2
;=====
addiu $t0, $0, a2
addiu $t1, $0, b2
add $t2, $t0, $t1
sll $t2, $t2, 8
or $s2, $s2, $t2
;=====
addiu $t0, $0, a3
addiu $t1, $0, b3
add $t2, $t0, $t1
sll $t2, $t2, 8
or $s2, $s2, $t2
```

7.6 Single Instruction Multiple Data

- Performing packed arithmetic requires modifying the ALU to eliminate carries between the smaller data elements. The carry out of (a0 + b0) should not affect the result of (a1 + b1).
- Short data elements often appear in **graphics processing**. For example, a pixel in a digital photo may use 8 bits to store each of the red, green, and blue color components.
- Using an entire 32-bit word to process one of these components wastes the upper 24 bits. When the components from four adjacent pixels are packed into a 32-bit word, the processing can be performed four times faster.
- SIMD instructions are even more helpful for **64-bit architectures**, which can pack eight 8-bit elements, four 16-bit elements, or two 32-bit elements into a single 64-bit word.
- SIMD instructions are also used for **floating-point computations**; for example, four 32-bit single-precision floating-point values can be packed into a single 128-bit word.

7. Advanced Microarchitecture

7.1 Deep Pipelines

7.2 Branch Prediction

7.3 Superscalar Processor

7.4 Out-of-Order Processor

7.5 Register Renaming

7.6 Single Instruction Multiple Data

7.7 Multithreading

7.8 Homogeneous Multiprocessors

7.9 Heterogeneous Multiprocessors

7.7 Multithreading

- **Multithreading**: a technique that helps a processor to utilize more execution units even if the ILP of a program is low or the program is stalled by switching to another thread.
- Computers can run multiple processes (running programs) simultaneously.
  Example: you can play music on a PC while surfing the web and running a virus checker.
- Each process consists of one or more threads that also run simultaneously.
- Example:
  - a word process may have one thread handling the user typing, a second thread spell-checking the document while the user works, and a third thread printing the document.
  - User does not have to wait for a document to finish printing before being able to type again.
- **The thread level parallelism (TLP)**: the degree to which a process can be split into multiple threads that can run simultaneously.

7.7 Multithreading

Conventional processor:

- Threads only give the illusion of running simultaneously by taking its turns for execution by the OS.
- When one thread's turn ends, the OS saves its architectural state, loads the architectural state of the next thread, and starts executing that next thread (**Context Switching**).
- As the processor switches through all the threads fast enough, the user perceives all the threads are running at the same time.

Multithreaded processor:

- It contains more than one copy of its architectural state, so many thread can be active at a time.
- MIPS processor can be extended to have four program counters and 128 registers.
  Then, four threads could be available at one time.
- If one thread stalls while waiting for data from main memory, the processor could context switch to another thread without any delay, because the program counter and registers are already available.

7.7 Multithreading

- If one thread doesn’t use all execution units, another thread could issue instructions to idle units.
- Multithreading does not improve the performance of an individual thread, because it does not increase the ILP.
- It does improve the overall **throughput** of the processor, because multiple threads can use processor resources that would have been idle when executing a single thread.
- Multithreading is also relatively inexpensive to implement, because it replicates only the PC and register file, not the execution units and memories.

7. Advanced Microarchitecture

7.1 Deep Pipelines

7.2 Branch Prediction

7.3 Superscalar Processor

7.4 Out-of-Order Processor

7.5 Register Renaming

7.6 Single Instruction Multiple Data

7.7 Multithreading

7.8 Homogeneous Multiprocessors

7.9 Heterogeneous Multiprocessors

7.8 Homogeneous Multiprocessors

- A multiprocessor system consists of multiple processors and a comm method btw the processors.
- Homogeneous (symmetric) multiprocessing (SMP) is a common form which two or more identical processors share a single main memory.
- The multiple processors may be separate chips or multiple cores on the same chip.
- Around 2005, computer architects made a major shift to building multiple copies (cores) of the processor on the same chip.

- Multiprocessors can be used to run **more threads simultaneously** or to run **one thread faster**.
- **Running more threads simultaneously** is easy; the threads are simply divided up among the processors. Unfortunately, typical PC users need to run only a small number of threads at any given time.

7.8 Homogeneous Multiprocessors

- **Running a particular thread faster** is much more challenging. The programmer must divide the existing thread into multiple threads to execute on each processor, which is tricky to let the processors communicate with each other.

- Other forms of multiprocessing include **heterogeneous** multiprocessing and **clusters**.
- **Heterogeneous multiprocessors** (asymmetric multiprocessors) use separate specialized microprocessors for separate tasks.
- In **clustered multiprocessing**, each processor has its own local memory system. Clustering can also refer to a group of PCs connected on the network running software to solve a large problem.

7. Advanced Microarchitecture

7.1 Deep Pipelines

7.2 Branch Prediction

7.3 Superscalar Processor

7.4 Out-of-Order Processor

7.5 Register Renaming

7.6 Single Instruction Multiple Data

7.7 Multithreading

7.8 Homogeneous Multiprocessors

7.9 Heterogeneous Multiprocessors

7.9 Heterogeneous Multiprocessors

- Heterogeneous multiprocessors merge different types of cores and/or specialized hardware in a single system.
- A heterogeneous system can merge cores with different microarchitectures that have different power, performance, and area tradeoffs.
- For example, a system could include both simple single-issue in-order cores and more complex superscalar out-of-order cores.
- The more power hungry out-of-order cores can be used to run applications that can make the higher performing, while the more energy efficient in-order cores can be used to the other applications that do not use the added computation effectively.
- Heterogeneous cores may all use the same ISA, which allows an application to run on any of the cores, or may employ different ISAs, which specify a core to a given task.

7.9 Heterogeneous Multiprocessors

- Heterogeneous systems can include a mix of traditional cores and specialized hardware.
- Example: Today's microprocessors include one or more floating-point units on chip and are now beginning to include other types of specialized hardware.
- Example: AMD and Intel both have processors that incorporate a graphics processing unit (GPU) or FPGA and one or more traditional x86 cores on the same chip.
- In power constrained environments, these types of integrated, specialized hardware provide better power-performance tradeoffs than performing the same operation on a standard core.

7.9 Heterogeneous Multiprocessors

Drawbacks of Heterogeneous:

- They add **complexity**, both in terms of designing the different heterogeneous elements and in terms of the additional programming effort to decide when and how to make use of the varying resources.

- **Homogeneous** multiprocessors are good for large data centers, that have lots of thread level parallelism available.
- **Heterogeneous** systems are good for cases that have more varying workloads and limited parallelism.

Assignment #5

Assignment #5

- Implement the coprocessor0 to handle exceptions, considering its instructions in *Appendix B*.
- Apply 2-bit dynamic branch prediction for branches, verifying using this shown code (Test#7).
- Do Synthesis & PnR and document all reports.
- Do Simulation after Implementation (PnR).
- Integrate coprocessor0 and branch predictor to MIPS project.
- Deadline: Friday, 29th Nov 2024, at 11:59 PM.
- Deliveries: Full design, testbench, a document and a presentation for your work.
  Send a mail to: eng.abdulkareem.abotaleb@gmail.com
  Using a title of: CompArch_assignment4_<first_name>_<last_name>

```
int c;
int main () {
    int i, j;
    for (i=0; i<1000; i++) {
        for (j=0; j<4; j++) {
            c++;
        }
    }
    return c;
}
```